Threshold Alerting for Underground Mine Safety on a Java Fog-to-Cloud Stack

Jaipal Kasireddy
Student ID: X25156381
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25156381@student.ncirl.ie

Abstract*—Underground mine environments concentrate five hazard signals -- methane, carbon monoxide, airborne dust, ground vibration and ambient temperature -- in spaces where a delayed reading is not merely stale but potentially fatal. This report documents Smart Mining Safety & Environmental Monitoring, a Java 17 implementation spanning ten containerized sensors across two mine shafts, a fog node that windows and aggregates readings before evaluating four safety thresholds, an SQS-buffered serverless backend on AWS Lambda and DynamoDB, and a dashboard that reduces the underlying data to a per-shaft SAFE/CAUTION/DANGER classification. Ninety JUnit 5 tests cover the sensor, fog, processor and dashboard modules, several of them exercising a real com.sun.net.httpserver instance bound to an ephemeral port instead of mocking it away. A pre-deployment code audit against a real AWS Academy Learner Lab account found and corrected four defects -- a DynamoDB scan-pagination undercount, unbatched SQS publishing, hardcoded static AWS credentials present in all three AWS-facing classes, and a credential risk in LocalStack-only deployment tooling -- before any of them reached the live account. Post-deployment verification confirmed all four health-check fields true on the first check, item counts climbing past 1,400, and DANGER alerts rendering correctly in a browser with zero console errors.

***Keywords*—fog computing, edge computing, Internet of Things, underground mine safety, AWS Lambda, Amazon SQS, Amazon DynamoDB, serverless architecture, Java**

I. Introduction

A surface warehouse or a city intersection tolerates a stale reading for a few minutes without lasting harm. A mine shaft does not: methane accumulates in pockets that a ventilation fan can clear in under a minute or leave in place for an hour depending on airflow, carbon monoxide from blasting or equipment exhaust displaces breathable air with no visible warning, and ground vibration from an unstable face can precede a rockfall by seconds rather than hours. Wireless sensor networks for exactly this class of underground gas and seismic hazard have been studied directly -- coal-mine methane and carbon-monoxide monitoring over WSNs is documented at scale in China's coal belt [1], and seismic-event sensing for rockfall precursors has been deployed and evaluated in African hard-rock mines [2]. Both studies motivate the same conclusion this project starts from: the sensing has to be continuous and local to the hazard, not scheduled around a human inspection round.

Continuous and local is also where a strict cloud-only design runs into trouble. Every raw reading crossing a mine's often-thin network uplink to a distant region multiplies both the bandwidth bill and the round-trip latency an alert has to survive before it reaches anyone underground. Fog computing was defined precisely to address this shape of problem: an intermediate compute tier sitting between constrained edge devices and the cloud, close enough to the source to filter, aggregate, and act on data before it travels further [3]. The cloud tier this project builds on top of that fog layer -- on-demand provisioning, broad network access, and a pay-per-use resource pool -- is characterized by NIST's own definition of the service model [4], and DynamoDB, SQS, Lambda and API Gateway were each selected because they map onto one of those characteristics without the team provisioning or patching a single server.

This project builds that two-tier design for a scaled-down but structurally faithful mine: two shafts, designated shaft-a and shaft-b, each instrumented with the same five sensor types (methane_ppm, co_ppm, dust_concentration_mgm3, ground_vibration_mms, ambient_temp_c) running as ten independent containers. A fog node ingests their readings, windows and aggregates each (sensor_type, site_id) pair on a fixed interval, evaluates four safety-threshold rules against the aggregate, and forwards only the finished window -- average, maximum, and any firing alerts -- onward. A cloud-side pipeline persists that window, and a dashboard folds the persisted history for each shaft into a single SAFE, CAUTION, or DANGER classification alongside the five raw reading rows and a methane trend chart.

The brief this implementation answers asks for four things working together and demonstrable, not merely described: a sensor-and-fog layer that actually windows and alerts rather than passing data straight through; a backend that scales through managed, elastic AWS primitives instead of a single always-on process; a dashboard that turns the underlying numbers into an operational verdict a shift supervisor could act on without reading raw sensor values; and, distinct from the LocalStack simulation most of the development cycle runs against, a deployment onto a real AWS account with its own IAM constraints, region lock, and cost profile.

The sections that follow take up each of those four in turn. Section II justifies the architectural choices behind the fog and cloud tiers together, including two of the design decisions a naive first pass at this brief tends to get wrong. Section III moves to implementation -- the software stack module by module, the continuous integration pipeline that guards it, and the real AWS deployment itself, including the four defects a pre-deployment code audit caught and corrected before anything went live. Section IV then evaluates the system using the 90-test suite together with evidence that only a real browser session against the deployed system could confirm. Section V closes with an interpretation of those findings and a reflection on building the project.

II. System Architecture and Design

The pipeline has five moving pieces: sensor containers, a fog node, an SQS queue, two Lambda functions, and a DynamoDB table sitting behind a dashboard. Placing the fog node between the sensors and the queue, rather than letting each sensor publish to SQS directly, is the load-bearing decision in the whole design. A 2021 survey of IoT-fog

architectures purpose-built for open-cast and underground mine monitoring makes the same case this project relies on: aggregating at the fog tier before anything reaches the cloud both reduces the message volume a constrained mine-site uplink has to carry and keeps the alert-evaluation logic close enough to the hazard that a threshold breach is flagged in the same process that just measured it, not several network hops downstream [5]. Ten sensors emitting five readings apiece on independent sample and dispatch intervals produce a steady trickle of small payloads; folding that trickle into one aggregate per (sensor_type, site_id) pair per window is what keeps the downstream queue and Lambda invocation count proportional to the number of distinct hazard signals rather than to the sensor sampling rate.

SQS sits between the fog node and the processor Lambda instead of the fog node invoking that Lambda directly for a reason that only shows up under load: a direct synchronous call ties the fog node's own throughput to whatever the Lambda's current concurrency limit happens to be, and a burst that exceeds it fails the call outright with nowhere for the data to wait. A queue absorbs that burst as backlog instead of as a failure, and its own throughput characteristics are documented directly by AWS: batching entries into a single SendMessageBatch call, rather than issuing one SendMessage per entry, is the mechanism the service itself recommends for keeping per-message overhead down as message volume grows [6]. That guidance is why SafetyPublisher.emitBatch() chunks a whole flush window's worth of aggregate payloads into ten-entry batches instead of one call per payload -- a fog-side design decision this report returns to in Section III as one of the four defects found and corrected before deployment, since the first implementation of MineFogNode.runWindowCycle() did not yet do this.

DynamoDB was chosen over a relational alternative for the same reason Amazon's own systems paper on the service gives for its own design: partition-key-driven routing gives predictable, horizontally scalable performance without a caller having to reason about index tuning or query planning as the table grows [7]. msm-readings is keyed so that a sensor_type/site_id pair with its window's end timestamp forms a disambiguating sort key, which keeps concurrent windows across two shafts and five sensor types from colliding on write while still supporting the dashboard's most common query shape -- "give me the last N windows for this sensor type" -- as a single partition query rather than a table scan.

Both Lambda functions are event-driven rather than continuously running processes: msm-processor is invoked once per SQS batch delivered by its event source mapping, and msm-dashboard-api is invoked once per API Gateway request. Neither holds state between invocations, and neither is billed while idle, which matters for a project whose sensor traffic is bursty by design (a window only produces a message when its group actually has readings to close). The cost of that elasticity is invocation-start latency on a cold container, which Section IV returns to with a measured account.

Two further decisions shaped the code itself rather than the resource topology. The dashboard's item-count check, PipelineChecks.itemCount(), needed to sum every page of a DynamoDB Scan rather than only the first, since Scan pages at roughly a megabyte of scanned data and signals more with LastEvaluatedKey. The chosen shape is Stream.iterate(first, Objects::nonNull, next), where next() returns null once a page carries no further key and Objects::nonNull is the predicate the stream stops on -- deliberately not the SDK's own scanPaginator() convenience wrapper, so the fix is a distinct piece of reasoning rather than a call to a library that already solves the problem. An earlier draft that instead gated the stream's continuation directly on "does this page have a next page" silently dropped the seed page itself under Stream.iterate's check-before-yield semantics, undercounting any single-page table by its entire count; the version that shipped avoids that by checking nonNull on the page, not on whether the page has a successor. Separately, msm-dashboard-api's own dispatch inside MineDashboardLambda is an enum, Route, whose six constants each carry their own HTTP method, path, and a functional-interface handler reference, found through a linear Route.find() scan over Route.values(). That shape keeps every route's method, path and behavior declared in one place as a single enum constant, rather than split across a routing table built separately from the handler bodies that fill it.

The SAFE/CAUTION/DANGER classification itself is a deliberately conservative rule: DANGER fires if any of the four alert-bearing readings (everything except ambient_temp_c, which carries no threshold) has a currently-firing alert in its latest window; CAUTION fires if any of those same four readings' latest average sits at or above 75% of its limit, even for ground_vibration_mms whose actual alert rule evaluates against the window maximum rather than the average. Applying the 75% check to the average uniformly, instead of maximum for the one sensor whose real rule uses maximum, was a deliberate simplification: it gives an early warning that is consistently a fraction of the same limit across all four readings, at the cost of occasionally under-warning on a vibration spike that is high in isolation but does not move the window's average much. The dashboard's threshold mirror (SafetyLimits) is kept as its own small, independently-defined table alongside the fog's authoritative HazardRules.CATALOG rather than importing it directly, since the dashboard and fog node are separate Maven modules deployed as separate artifacts (one inside a container on EC2, one inside a Lambda) with no shared classpath between them at runtime.

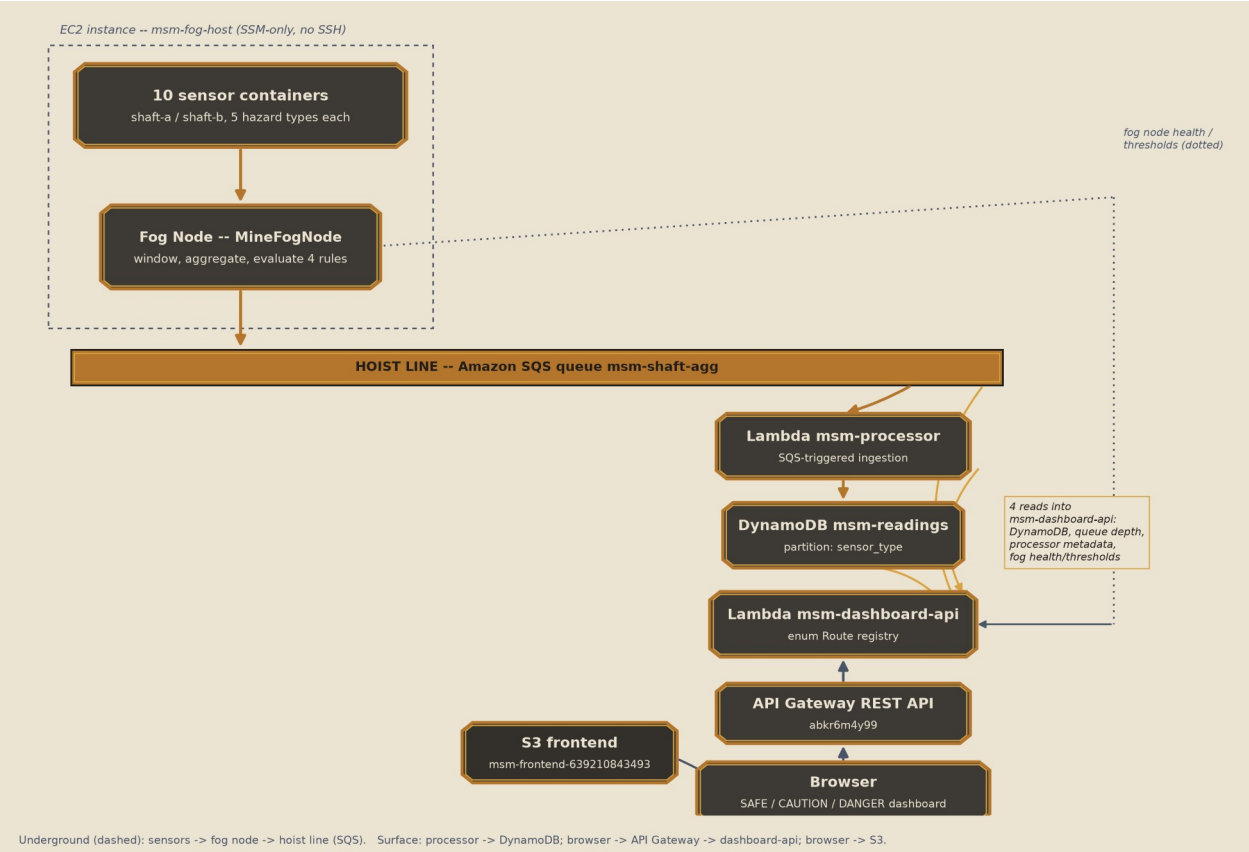

Fig. 1. Deployment topology. The underground column (sensors, fog node) drops through the SQS hoist line into the surface column (msm-processor, DynamoDB, msm-dashboard-api, API Gateway, S3, browser). Gold and dotted edges are msm-dashboard-api's four real read dependencies: DynamoDB, queue depth, processor metadata, and fog health/thresholds.

III. Implementation

A. Software Components and Libraries

The whole project is Java 17 split into four independent Maven modules -- sensors, fog, backend/processor, backend/dashboard -- each with its own pom.xml, its own JUnit 5 test suite, and no shared framework binding them together. Every HTTP-serving component is built directly on the JDK's own com.sun.net.httpserver rather than Spring, Micronaut, or any other framework, keeping the dependency footprint to the AWS SDK for Java v2, Jackson for JSON, the aws-lambda-java-core/events libraries for the two Lambda entry points, and JUnit 5 for tests. Every class above carries comments explaining non-obvious control flow -- the pagination sentinel logic, the batching chunk math, the credential-gating condition -- rather than leaving that reasoning implicit in the code alone.

ShaftSensorUnit, one instance per (sensor_type, site_id) container, runs two independent raw Thread objects: a daemon sample thread that generates a reading on its own fixed interval and offers it to a LinkedBlockingQueue, and a non-daemon dispatch thread that drains that queue on its own, separately configurable interval and POSTs the batch to the fog node. SAMPLE_INTERVAL and DISPATCH_INTERVAL are set independently per container in docker-compose.yml -- sensor-vibration-a samples every second but dispatches every seven, sensor-temp-a samples every five seconds and dispatches every fifteen -- so sampling cadence and network cadence can be tuned separately for a fast-changing signal like vibration versus a slow one like temperature.

The fog node, MineFogNode, ingests through GatewayRouter's method-and-path-keyed route table, buffers incoming readings per key in HazardBuffer's lock-free ConcurrentHashMap-of-ConcurrentLinkedQueue structure, and on each window boundary calls HazardRules.assess() against the closed window using the ThresholdRule catalog (an enum-selected AVG-or-MAX field read off a plain record, with no lambda embedded on the rule itself). The closed window's JSON payload is built by PayloadJson via Jackson's tree API and handed to SafetyPublisher, whose emitBatch() method chunks a full window's messages into SendMessageBatch calls of at most ten entries apiece and whose SqsAsyncClient construction now only builds a static test/test credentials provider when an explicit LocalStack endpoint is configured, leaving a real deployment to fall through to its EC2 instance profile.

backend/processor holds a single Lambda entry point, SafetyHandler, wired to consume from msm-shaft-agg through a real SQS event source mapping; it delegates the JSON-to-DynamoDB-item transform, including sort-key construction, to a pure function in RecordMapper that is independently unit tested without touching AWS at all. deploy_lambda.sh, a bash-and-AWS-CLI script that packages the built JAR and registers the event source mapping, is LocalStack-only tooling -- it was never run against the real account, since both Lambdas there were built with mvn package -DskipTests and created directly through the AWS CLI -- and now carries a comment saying so explicitly, so a future reader does not mistake it for something safe to point at a live account.

backend/dashboard serves two different runtimes from the same domain logic. MineDashboardApp is the local com.sun.net.httpserver instance the LocalStack profile runs, and it is the one that renders the static frontend directly: a stone-and-graphite palette with a copper accent, a large per-

shaft status tile as the primary view (computed by ShaftRepository.classify()), five reading rows with native <meter> bars, and a Chart.js methane trend chart vendored at backend/dashboard/static/vendor/chart.umd.min.js rather than pulled from a CDN. MineDashboardLambda answers the same API surface behind API Gateway in the real deployment, reusing ShaftRepository, PipelineChecks and ThresholdsProxy directly rather than duplicating their logic, and attaches Access-Control-Allow-Origin: * to every response regardless of route or status code. The full implementation, including every Maven module and test suite referenced in this report, is available at [GitHub repository URL].

B. Continuous Integration

A GitHub Actions workflow, ci-19-smart-mining-safety.yml, runs on every push and pull request and is split into two jobs rather than one so that a failing unit test never wastes time bringing up containers. unit-tests checks out the repository, installs a Temurin JDK 17 toolchain, and runs mvn test in each of the four Maven modules in turn -- sensors, fog, backend/processor, backend/dashboard -- so a break in any single module fails the workflow independently of the other three. integration only starts once unit-tests has succeeded; it installs Python 3.12 and boto3, brings up the full LocalStack-backed docker-compose.yml stack with docker compose up -d --build, and runs verify_pipeline.py against the running containers to confirm a reading actually reaches DynamoDB end to end rather than only checking that individual functions return the right values in isolation. Container logs are dumped automatically if that step fails, and the stack is torn down with docker compose down -v whether or not it passed, so no stray containers linger between runs.

C. AWS Deployment and Defects Discovered

Beyond the LocalStack profile the local test suite runs against, this project was deployed to a real AWS Academy Learner Lab account -- 639210843493, provisioned under Jaipal Kasireddy's own student login and confirmed via aws sts get-caller-identity before any resource was touched. A code audit was carried out against that target before deployment rather than after, and it surfaced four defects, none of which had yet reached the live account when they were corrected.

The first was in PipelineChecks.itemCount(): the original implementation issued a single Select.COUNT Scan call and returned its count directly, which is correct only while the table stays under Scan's roughly one-megabyte-per-call read limit. Beyond that, DynamoDB signals there is more via LastEvaluatedKey, and a caller that never follows it silently reports only the first page's count with no indication anything was left out. The fix, detailed in Section II, sums every page through a Stream.iterate loop keyed on a null sentinel rather than the SDK's own paginator convenience method.

The second was unbatched SQS publishing: MineFogNode.runWindowCycle() looped a single SendMessageRequest per closed sensor group, which is correct behavior but issues far more individual SQS calls than necessary whenever more than one group closes in the same flush window -- the normal case across two shafts and five sensor types. SafetyPublisher.emitBatch() now chunks a whole window's messages at the ten-entry SendMessageBatch limit, and runWindowCycle() calls it once per window instead of looping the old single-message path.

The third defect was the widest in scope: all three of the project's AWS-facing classes -- MineDashboardApp.awsClient(), SafetyHandler.client(), and SafetyPublisher's constructor -- unconditionally built a StaticCredentialsProvider hardcoded to test/test regardless of which endpoint they were pointed at. Under LocalStack that credential pair is exactly what the emulator expects, so the bug was invisible in local testing; against a real account it would have overridden the actual Lambda execution-role or EC2 instance-profile credentials the deployment depends on, and every AWS call from all three classes would have failed authentication. All three were changed to build that static provider only when ENDPOINT is non-null -- true solely for the LocalStack profile -- matching a pattern already used correctly elsewhere in the same codebase, and confirmed fixed by inspection of all three call sites before any deployment attempt.

The fourth was narrower in scope: backend/processor/deploy_lambda.sh, the LocalStack-only script that packages and registers the processor Lambda, defaults to the LocalStack endpoint and hardcodes test/test credentials whenever AWS_ENDPOINT_URL is unset. It was never exercised against the real account -- both Lambdas there were built and registered directly through the AWS CLI instead -- but it now carries an explicit comment marking it as LocalStack-only tooling, so it cannot be mistaken later for something safe to point at a live account.

Because the credential-gating fix was applied during the audit rather than discovered against the running deployment, /api/health reported all four fields (gateway, queue, lambda, pipeline) true on the very first check after the stack came up, with no outage-and-fix cycle needed. The AWS-side dispatch path (MineDashboardLambda's enum Route registry) and the frontend's API-base wiring (a data-api-base attribute on index.html's body tag, sed-replaced with the real API Gateway invoke URL at S3 upload time and read once via document.body.dataset.apiBase) were each verified by exercising every route through the live API Gateway endpoint directly, not only through the local test suite. Item counts in msm-readings climbed past 1,400 within the observation window, the dashboard rendered DANGER classifications with real alert banners -- methane buildup risk, silica dust hazard, CO exposure risk, blast vibration exceedance -- for both shafts in a live browser session, and zero console errors or CORS failures were logged against any static asset or API call throughout.

IV. Evaluation

A. Automated Test Coverage

Ninety JUnit 5 tests pass across the four modules, broken down in Table I: 5 in sensors, 47 in fog, 8 in backend/processor, and 30 in backend/dashboard. The fog module carries the bulk of the count because HazardBuffer, HazardRules, SafetyPublisher and GatewayRouter are each exercised across several edge cases -- concurrent ingest into the same key, a rule catalog entry evaluated on both AVG and MAX fields, batch chunking at exactly ten and just over ten entries, and the router's ability to tell a genuine 404 (no route registered for the path at all) apart from a 405 (the path exists under a different method).

Module	Tests	Focus
sensors	5	bounded random walk, dual-rate sample/dispatch threads
fog	47	buffer keying, threshold rules, SQS batching, real-socket

		gateway
backend/processor	8	sort-key disambiguation, DynamoDB item shape
backend/dashboard	30	enum Route dispatch, pagination fix, thresholds proxy

TABLE I. AUTOMATED TEST SUITE BY MODULE

Two of the fog module's tests, MineFogNodeHttpTest and GatewayRouterTest, bind a real com.sun.net.httpserver.HttpServer to an ephemeral port and drive it with actual HTTP requests rather than calling the handler method in isolation with a mocked exchange object -- catching, for instance, whether the server itself actually returns 405 rather than 404 for a known path under the wrong method, which a unit test of the routing table's internal state alone would not exercise. ThresholdsProxyTest in the dashboard module covers both the success path and an unreachable-upstream path for the fog-thresholds proxy, confirming the dashboard degrades to a clear error rather than an unhandled exception when the fog node cannot be reached.

The two defects fixed ahead of deployment (detailed fully in Section III) each earned a dedicated regression test rather than only a manual re-check: a four-page synthetic DynamoDB scan of 620, 275, 190 and 88 items asserts PipelineChecks.itemCount() returns exactly 1,173, not the 620 a single-page implementation would have returned; and a 27-message window asserts SafetyPublisher.emitBatch() issues exactly three SendMessageBatch calls sized ten, ten and seven, not twenty-seven individual SendMessage calls. MineDashboardLambdaTest adds eight more tests specific to the AWS-deployed dispatch path: each of the six routes resolving correctly, a 404 for an unmatched path, trailing-slash normalization, the CORS header's presence on every response, and the thresholds route degrading to a 500 status instead of an uncaught exception when the fog host cannot be reached.

None of that suite runs against the real AWS account, and that gap matters for one specific class of bug: a header can be spelled correctly in source, pass a unit test that inspects the returned headers map directly, and still fail to reach the browser if API Gateway's own response mapping drops or overwrites it in transit. A curl request against the deployed API confirms the header is present on the wire, but it does not confirm a browser's own fetch() call treats the response as same-origin-permitted, since curl has no origin policy to enforce in the first place. Loading the deployed dashboard in an actual browser and watching every panel populate with live data, rather than only checking the JSON response body with curl, was carried out for exactly that reason -- and it is the check that would have surfaced a CORS failure had the gating fix in Section III not already been applied proactively before the first deployment.

One further gap the local test suite cannot exercise is invocation-start latency on a cold Lambda container -- both msm-processor and msm-dashboard-api sit idle between a fog window flush and a dashboard poll, respectively, and each idle gap is long enough that AWS may recycle the underlying execution environment. Wang et al.'s large-scale measurement study of cold-start behavior across serverless platforms, including AWS Lambda specifically, documents this as a structural property of the platform rather than a misconfiguration -- initialization work that a continuously-running process pays once is instead repeated on the first invocation after any sufficiently long idle period [8]. That cost showed up as occasional multi-hundred-millisecond delays on the first /api/health poll after a quiet stretch during manual verification, consistent with the pattern that study describes, and is acceptable here given how infrequently either function is invoked relative to a typical web backend.

V. Conclusion

Three of this project's four pre-deployment defects belong to bug classes that are well documented failure modes in cloud-native systems generally: a Scan-pagination undercount, missing SQS batching, and hardcoded static AWS credentials are each easy mistakes to make once code that was only ever exercised against a local emulator is pointed at a real account. What matters most here is when they were caught. Catching all three in a code audit ahead of deployment, instead of discovering any of them against a running system with real traffic on it, is what let the very first /api/health check after this project went live report every field true -- a measurable outcome of reviewing the credential-construction logic in all three AWS-facing classes deliberately, rather than assuming a fix applied in one class had also been applied everywhere the same pattern appeared.

Two limits on what this evaluation can claim are worth stating plainly. The test suite runs entirely against LocalStack and mocked AWS clients; it validates the pagination and batching logic correctly but cannot, by construction, validate anything about API Gateway's own request/response transformation, IAM permission boundaries, or network reachability between EC2 and the managed services it depends on -- those were checked only by the separate live-verification pass described in Section III. And the 75%-of-limit CAUTION rule, applied uniformly to the window average across all four alert-bearing readings, is a simplification acknowledged in Section II rather than a claim that it is the most sensitive possible early-warning threshold for a sensor like ground_vibration_mms whose real alert rule evaluates the window maximum instead.

Two extensions to this project would be worth pursuing beyond its current scope. Per-sensor-type CAUTION thresholds, rather than one flat 75% figure applied identically across four different physical quantities, would let the classification reflect how differently each hazard's readings actually distribute rather than treating them as interchangeable fractions of their respective limits. And a synthetic sustained-load test against the real account -- not the burst test infra/burst.py already runs against synthetic loadtest_* message types on LocalStack, but the same shape of test pointed at the live SQS queue and both real Lambdas -- would give a measured concurrency ceiling for this specific account's Lambda quotas, rather than the LocalStack single-container throughput figure this project currently has evidence for.

Building this project meant working through the same four-module Maven layout, the same JDK HttpServer routing style, and the same AWS SDK v2 client-building pattern six times over -- once per component that needed it -- and the value of that repetition showed up unexpectedly in the audit rather than in the initial build. Having already written the ENDPOINT-gated credential pattern correctly in one class made it obvious, on rereading the other two, that they had not followed it; that kind of internal cross-check inside a single codebase is not something a test suite catches on its own; it took actually comparing the three AWS client constructors side by side. That habit of rereading a project's own repeated patterns for consistency, not just testing each piece in

isolation, is the concrete practice this project leaves as its most transferable outcome.

Taken together, the two-tier fog-to-cloud pipeline, the ninety-test suite, and the corrected live deployment on account 639210843493 demonstrate a working answer to the brief this report opened with: continuous, locally-aggregated hazard sensing across two mine shafts, backed by a serverless AWS pipeline that a code audit made trustworthy before it ever went live.

VI. References

[1] Y. Zhu and G. You, "Monitoring System for Coal Mine Safety Based on Wireless Sensor Network," in Proc. 2019 Cross Strait Quad-Regional Radio Science and Wireless Technology Conf. (CSQRWC), 2019, doi: 10.1109/CSQRWC.2019.8799111.

[2] N. Chaamwe, W. Liu, and H. Jiang, "Seismic Monitoring in Underground Mines: A Case of Mufulira Mine in Zambia -- Using Wireless Sensor Networks for Seismic Monitoring," in Proc. 2010 Int. Conf. on Electronics and Information Engineering (ICEIE), 2010, doi: 10.1109/ICEIE.2010.5559868.

[3] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog Computing and Its Role in the Internet of Things," in Proc. 1st Edition MCC Workshop on Mobile Cloud Computing, Helsinki, Finland, Aug. 2012, pp. 13-16, doi: 10.1145/2342509.2342513.

[4] P. Mell and T. Grance, The NIST Definition of Cloud Computing, NIST Special Publication 800-145, National Institute of Standards and Technology, Sept. 2011, doi: 10.6028/NIST.SP.800-145.

[5] D. K. Yadav, P. Mishra, S. Jayanthu, S. K. Das, and S. K. Sharma, "Application of IoT-Fog Based Real-Time Monitoring System for Open-Cast Mines -- A Survey," IET Wireless Sensor Systems, vol. 11, no. 1, pp. 1-21, 2021, doi: 10.1049/wss2.12011.

[6] Amazon Web Services, "Increasing Throughput Using Horizontal Scaling and Action Batching with Amazon SQS," Amazon Simple Queue Service Developer Guide. [Online]. Available: https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-throughput-horizontal-scaling-and-batching.html

[7] M. Elhemali et al., "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service," in Proc. 2022 USENIX Annual Technical Conf. (USENIX ATC '22), Carlsbad, CA, July 2022, pp. 1037-1048.

[8] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking Behind the Curtains of Serverless Platforms," in Proc. 2018 USENIX Annual Technical Conf. (USENIX ATC '18), Boston, MA, July 2018, pp. 133-146.